

Compilador HULK

Reporte del Proyecto de Compilación y Lenguajes de Programación

Integrantes:

Aixa Bazán Rodríguez
Lianet Soto Aguirre
Amanda Medina Solis

Repositorio del proyecto:

https://github.com/LianetSoto/Hulk_Compiler

Facultad de Matemática y Computación
Universidad de La Habana

Índice general

Introducción	2
1. Análisis del diseño del lenguaje HULK	3
1.1. Estructura general de un programa	3
1.2. Operaciones y tipos básicos	3
1.3. Expresiones y control de flujo	4
1.3.1. Bloques de expresiones	5
1.3.2. Condicionales	5
1.3.3. Bucles	5
1.3.4. Variables y ámbito	6
1.4. Funciones	7
1.5. Tipos	9
1.6. Sistema de tipos	12
1.6.1. Tipado estático e inferencia de tipos	12
1.6.2. Conformidad de tipos	12
1.6.3. Comprobación y conversión dinámica de tipos	13
2. Extensiones añadidas al lenguaje	15
2.1. Polimorfismo paramétrico en funciones	15
2.2. Protocolos	16
2.3. Iterables	17
3. Pipeline de compilación	20
3.1. Análisis léxico	21
3.2. Análisis sintáctico	22
3.3. Árbol de Sintaxis Abstracta	25
3.4. Análisis semántico	26
3.5. Monomorfización de funciones genéricas	29
3.6. Generación de código	30
3.7. Compilación a código nativo	34
3.8. Gestión de errores	34
Conclusiones	35

Introducción

HULK (Havana University Language for Kompilers) es un lenguaje de programación didáctico desarrollado en la Universidad de La Habana como parte del curso de Introducción a los Compiladores [1]. Se trata de un lenguaje estáticamente tipado, orientado a objetos y basado en expresiones.

Este documento constituye el reporte del proyecto realizado durante el curso de Compilación y Lenguajes de Programación. El trabajo desarrollado se articula en torno a dos grandes ejes. El primero de ellos consiste en un análisis crítico del diseño del lenguaje HULK, examinando sus principales decisiones sintácticas y semánticas, evaluando sus ventajas y limitaciones, y comparándolas con las soluciones adoptadas por otros lenguajes de programación. El segundo eje comprende la descripción de las extensiones incorporadas al lenguaje por el equipo de desarrollo, así como la arquitectura del compilador que le da soporte, abarcando desde el análisis léxico hasta la generación de código final.

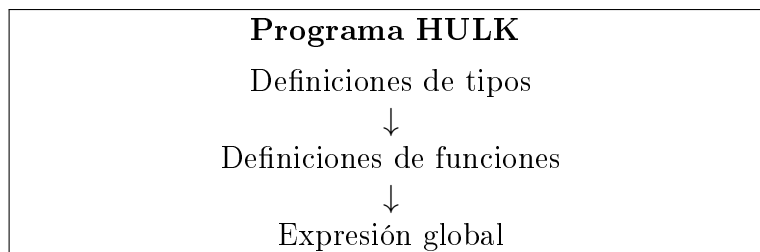
Capítulo 1

Análisis del diseño del lenguaje HULK

En este capítulo se presenta el lenguaje HULK base desde la perspectiva de sus principales decisiones de diseño. Más que describir exhaustivamente su sintaxis, el objetivo es analizar las características que definen su modelo de programación y compararlas con las soluciones adoptadas por otros lenguajes modernos cuando resulte pertinente.

1.1. Estructura general de un programa

Todo programa en HULK está formado por una secuencia de definiciones globales seguida de una única expresión que constituye el punto de entrada del programa. Las definiciones pueden corresponder a tipos o funciones, mientras que la expresión final representa el código que será evaluado al comenzar la ejecución.



1.2. Operaciones y tipos básicos

Como la práctica totalidad de los lenguajes de programación, HULK proporciona un conjunto de tipos predefinidos que constituyen los bloques elementales sobre los que se construye cualquier programa: **Number**, **String** y **Boolean**. Aunque a primera vista puedan parecer una elección trivial, un examen desde la definición formal de tipo revela aciertos y limitaciones que merecen ser discutidos.

Number

El tipo **Number** representa el conjunto de los números en coma flotante de 64 bits según el estándar IEEE 754, con la notación sintáctica habitual de dígitos y punto decimal opcional.

A diferencia de lenguajes como **C#**, donde existen múltiples tipos numéricos diferenciados (**int**, **float**, **double**, entre otros), HULK adopta un modelo unificado similar al de JavaScript, en el que todos los valores numéricos se representan con un único tipo

en coma flotante. Esto simplifica el sistema de tipos y reduce la complejidad tanto para el programador como para el diseño del lenguaje, aunque ello implica perder precisión exacta en cálculos enteros y control fino sobre la representación numérica.

Las operaciones definidas sobre este tipo son la suma (+), resta (-), multiplicación (*), división en coma flotante (/), potencia (^), módulo (%) y las subexpresiones entre paréntesis, todas ellas con las reglas usuales de precedencia y asociatividad.

En la especificación original de HULK, el tipo **Number** se define como un flotante de 32 bits. En nuestra implementación se utiliza precisión de 64 bits, lo que mantiene el mismo comportamiento semántico del lenguaje pero mejora la estabilidad numérica en cálculos generales.

String

El tipo **String** representa el conjunto de todas las secuencias finitas de caracteres, cuyos literales se definen mediante comillas dobles (") y admiten el uso de secuencias de escape estándar para representar caracteres especiales.

Las operaciones definidas sobre este tipo se limitan a la concatenación mediante los operadores @ y @@. El operador @@ difiere de @ en que inserta automáticamente un espacio entre los operandos. Además, ambos operadores permiten concatenar cadenas con valores numéricos, convirtiendo estos últimos de forma implícita a su representación textual cuando es necesario.

A diferencia de lenguajes como Java o Python, donde la concatenación se realiza mediante la sobrecarga del operador +, HULK introduce operadores específicos para evitar ambigüedades entre la suma aritmética y la concatenación de cadenas, simplificando así la semántica del lenguaje y haciendo explícita la intención de cada operación.

Boolean

El tipo **Boolean** representa el dominio de los valores booleanos, cuyos únicos literales son **true** y **false**. Sobre este tipo se definen los operadores lógicos & (and), | (or) y ! (not).

Asimismo, HULK incorpora diversos operadores cuyo resultado es un valor **Boolean**. Entre ellos se encuentran los operadores de comparación <, >, <= y >=, aplicables únicamente a valores de tipo **Number**, así como los operadores de igualdad == y !=, que pueden utilizarse sobre cualquiera de los tipos básicos (**Number**, **String** y **Boolean**). De este modo, el tipo **Boolean** constituye la base de la evaluación de expresiones condicionales y del control del flujo de ejecución en el lenguaje.

1.3. Expresiones y control de flujo

Tras introducir los tipos básicos, resulta necesario analizar una de las decisiones de diseño más representativas de HULK: su orientación a expresiones. Un lenguaje basado en expresiones es aquel en el que prácticamente todas sus construcciones sintácticas producen un valor y, por tanto, pueden utilizarse como parte de expresiones de mayor complejidad. En las siguientes secciones se analizará cómo esta filosofía influye en el diseño de las distintas estructuras de HULK, así como las ventajas y limitaciones que introduce.

1.3.1. Bloques de expresiones

En cualquier contexto donde se espere una expresión, HULK permite utilizar un bloque delimitado por llaves. Dicho bloque está formado por una secuencia de expresiones separadas por punto y coma, cuyo valor final es el de la última evaluada. Esta construcción permite agrupar varias operaciones sin abandonar el modelo orientado a expresiones del lenguaje.

1.3.2. Condicionales

La construcción condicional en HULK se implementa mediante la expresión `if`. La condición debe ser una expresión de tipo `Boolean` y el lenguaje admite múltiples ramas mediante la construcción `elif`. Asimismo, tanto la rama `if` como las ramas `elif` y `else` pueden estar formadas por una única expresión o por un bloque de expresiones.

Como consecuencia de la naturaleza orientada a expresiones de HULK, todo condicional debe producir un valor. Esto implica que la rama `else` es obligatoria, incluso en situaciones en las que el programador únicamente desea ejecutar una acción cuando la condición se cumple. Aunque esta restricción simplifica la semántica del lenguaje y evita caminos de ejecución sin valor de retorno, también puede dar lugar a ramas artificiales cuyo único propósito es satisfacer los requisitos sintácticos del lenguaje.

Esta obligatoriedad del `else` es compartida por otros lenguajes que también son orientados a expresiones como Haskell, por ejemplo. En lenguajes basados en sentencias como C, C++ o Java, el `else` es opcional porque el `if` no devuelve ningún valor y su única función es controlar el flujo de ejecución. Esta diferencia fundamental refleja la distinción entre lenguajes que tratan las estructuras de control como productoras de valores y aquellos que las tratan como meras secuencias de acciones.

1.3.3. Bucles

HULK incorpora dos mecanismos de iteración: `while` y `for`. El cuerpo de ambos bucles puede estar formado por una única expresión o por un bloque de expresiones.

Bucle `while`

El bucle `while` ejecuta repetidamente su cuerpo mientras la condición, de tipo `Boolean`, permanezca verdadera. Al tratarse de una expresión, el valor del bucle corresponde al de la última iteración ejecutada. Si la condición es falsa desde el inicio, el lenguaje devuelve un valor por defecto del tipo inferido.

Bucle `for`

Por su parte, el bucle `for` permite recorrer los valores generados por la función predefinida `range(inicio, fin)`, que produce la secuencia de enteros comprendidos entre `inicio` (incluido) y `fin` (excluido). En este aspecto, su diseño se aproxima al de Python, donde el bucle `for` también itera sobre los valores generados por `range`, en contraste con el modelo de C o C#, basado en una inicialización explícita, una condición de terminación y una expresión de actualización.

Esta aproximación reduce la posibilidad de errores asociados al manejo manual de la variable de control y favorece una sintaxis más sencilla y legible cuando el número de iteraciones es conocido.

Mientras que en Python el bucle `for` es capaz de iterar sobre cualquier objeto que implemente el protocolo de iteración, el diseño de HULK resulta más limitado al restringir su uso exclusivamente a la función `range`. Esta decisión simplifica la implementación del lenguaje y reduce la complejidad del sistema de tipos —ya que no requiere un mecanismo de iteradores genéricos—, pero también limita su expresividad en comparación con lenguajes que ofrecen una abstracción más general de la iteración.

Asimismo, el lenguaje no incorpora instrucciones como `break` o `continue`, presentes en C, Java, C# o Python. La ausencia de estas instrucciones obliga a modelar las salidas anticipadas o la omisión de iteraciones mediante condiciones adicionales en el cuerpo del bucle, lo que puede hacer que determinados algoritmos resulten menos naturales de expresar.

1.3.4. Variables y ámbito

Junto con la orientación a expresiones, el tratamiento de las variables constituye la decisión de diseño que más distingue a HULK del paradigma imperativo dominante. HULK adopta una sintaxis de declaración explícita, característica de numerosos lenguajes funcionales, como Haskell, OCaml o F#. Esta elección tiene consecuencias profundas en la legibilidad del código, la gestión de ámbitos y la relación entre mutabilidad y control del estado.

El `let` como expresión

En HULK, las variables se introducen mediante la expresión `let`, que crea un nuevo ámbito léxico delimitado por la palabra clave `in`. A diferencia de los lenguajes imperativos, donde la declaración de variables constituye una sentencia, `let` produce el valor de la expresión evaluada tras `in`. Esto permite emplear definiciones locales como parte de expresiones de mayor complejidad, sin necesidad de introducir variables temporales en ámbitos más amplios.

HULK permite declarar varias variables dentro de una misma expresión `let`, separadas por comas. No obstante, esta sintaxis constituye únicamente azúcar sintáctico sobre una secuencia de expresiones `let` anidadas, por lo que las definiciones son secuenciales y cada variable puede utilizar las previamente declaradas. Este diseño coincide con el de Haskell y difiere del mecanismo `let ... and ...` de OCaml, cuyas definiciones son paralelas y, por tanto, no pueden depender unas de otras.

Como consecuencia del alcance léxico, HULK permite redefinir un identificador en un ámbito interno sin modificar la asociación existente en ámbitos exteriores. Este mecanismo, conocido como *shadowing*, es habitual en lenguajes funcionales y modernos como Rust o Scala. Aunque facilita reutilizar identificadores cuando dejan de ser relevantes sus valores anteriores, un uso excesivo puede dificultar el seguimiento del flujo de datos.

Frente al modelo basado en sentencias de lenguajes como C o Python, la sintaxis `let ... in ...` puede resultar más verbosa en situaciones sencillas. Mientras que una variable temporal puede declararse y utilizarse de forma directa, en HULK es necesario delimitar explícitamente el ámbito mediante `in`, lo que introduce un mayor anidamiento y puede reducir la legibilidad en secuencias largas de definiciones locales.

Asignación destructiva: mutabilidad controlada

HULK no permite modificar una variable mediante la misma sintaxis con que se declara; para ello introduce el operador de asignación destructiva `:=`. Esta distinción explícita entre «definir» y «modificar» es infrecuente en los lenguajes de propósito general, donde el mismo operador suele emplearse tanto para la inicialización como para la asignación.

Dicha decisión hace explícitos los puntos del programa donde se producen efectos laterales, facilitando distinguir entre la introducción de nuevas variables y la modificación del estado. En Rust, por ejemplo, la mutabilidad se expresa mediante la anotación `let mut`, mientras que HULK la permite siempre, pero señala las actualizaciones mediante un operador específico. Ambos enfoques persiguen hacer visible la mutabilidad al lector del código, aunque HULK adopta una solución menos restrictiva al no requerir declarar previamente una variable como mutable.

El coste de esta decisión es una ligera sobrecarga sintáctica, que puede resultar innecesaria en programas sencillos. Sin embargo, en aplicaciones de mayor tamaño contribuye a mejorar la legibilidad del código y facilita el razonamiento sobre los cambios de estado.

La asignación destructiva también constituye una expresión y devuelve el valor recién asignado.

1.4. Funciones

HULK permite definir funciones propias mediante la palabra reservada `function`. Todas las funciones se declaran antes de la expresión global del programa y poseen un cuerpo formado por una única expresión o por un bloque de expresiones.

Forma	Sintaxis
Inline	<code>function f(x) =>expr;</code>
Completa	<code>function f(x) { ... }</code>

Tabla 1.1: Formas de declarar funciones en HULK.

La existencia de ambas sintaxis constituye una decisión de diseño equilibrada. La forma abreviada reduce la verbosidad en funciones simples, mientras que la forma completa es útil cuando el algoritmo requiere varios pasos intermedios.

Retorno implícito

Al igual que los condicionales, los bucles o los bloques de expresiones, las funciones mantienen la filosofía orientada a expresiones de HULK. El cuerpo de una función no está formado por una secuencia de sentencias que concluyen mediante una instrucción `return`, sino por una expresión cuyo valor constituye automáticamente el valor de retorno de la función.

Algunos lenguajes modernos adoptan un enfoque híbrido. Rust, por ejemplo, utiliza el valor de la última expresión como retorno implícito, aunque también permite emplear la instrucción `return` cuando se desea abandonar la función de forma anticipada o mejorar la claridad del código en determinados contextos.

El diseño adoptado por HULK reduce la carga sintáctica y mantiene un modelo uniforme de evaluación, ya que las funciones siguen las mismas reglas que el resto de las

expresiones del lenguaje. No obstante, esta decisión también restringe el estilo de programación. Al no existir retornos explícitos, la función no puede finalizar anticipadamente cuando se detecta un caso particular, como una condición de error o una validación. En consecuencia, el programador debe expresar toda la lógica mediante la composición de expresiones, lo que preserva la coherencia del modelo del lenguaje, pero puede dificultar la lectura de funciones con numerosos casos de salida.

Espacio de nombres y visibilidad

Todas las funciones definidas por el usuario pertenecen a un único espacio de nombres global. Como consecuencia, no es posible declarar dos funciones con el mismo identificador, independientemente del número o del tipo de sus parámetros.

Esta decisión simplifica considerablemente el proceso de resolución de llamadas durante la compilación, ya que cada identificador hace referencia de manera unívoca a una única función. Sin embargo, impide la existencia de sobrecarga de funciones, sacrificando parte de la flexibilidad que ofrecen otros lenguajes con la misma.

Otra consecuencia de este modelo global es que HULK tampoco incorpora mecanismos para restringir la visibilidad de las funciones. A diferencia de otros lenguajes, donde es posible distinguir entre funciones públicas y privadas mediante modificadores de acceso, en HULK todas las funciones son accesibles desde cualquier punto del programa y forman parte de la interfaz visible del lenguaje.

Esta decisión reduce el número de conceptos que el programador debe dominar y mantiene un modelo de organización muy sencillo, coherente con el carácter didáctico de HULK. Sin embargo, es una limitación que resulta especialmente significativa a medida que aumenta el tamaño del software, donde el encapsulamiento constituye uno de los mecanismos fundamentales para controlar la complejidad.

Orden de declaración

Las funciones pueden invocarse independientemente del orden en que aparecen en el código fuente. En consecuencia, una función puede llamar a otra cuya definición aparezca posteriormente en el programa.

Dicha decisión favorece la modularidad del código, ya que el programador no necesita organizar las definiciones siguiendo el orden de dependencia entre ellas. Como contrapartida, el compilador debe realizar una fase previa de recolección de todas las firmas antes de analizar los cuerpos de las funciones, incrementando ligeramente la complejidad de su implementación.

Funciones y constantes predefinidas

Además de las funciones definidas por el usuario, HULK incorpora un pequeño conjunto de funciones matemáticas predefinidas (`sqrt`, `sin`, `cos`, `exp`, `log` y `rand`), así como las constantes globales `PI` y `E`. Todas ellas forman parte del núcleo del lenguaje y pueden utilizarse directamente sin necesidad de importar bibliotecas adicionales.

Asimismo, HULK proporciona la función predefinida `print`, utilizada para mostrar el resultado de una expresión por la salida estándar. Esta función admite directamente valores de los tipos básicos `Number`, `String` y `Boolean`, lo que permite visualizar los resultados de los programas sin necesidad de mecanismos adicionales de conversión.

Esta decisión prioriza la simplicidad y reduce la carga cognitiva del programador, especialmente en un contexto donde resulta conveniente disponer desde el primer momento de las operaciones matemáticas más habituales y de un mecanismo básico de salida. Sin embargo, mantener estas funciones y constantes en el espacio global limita la modularidad del lenguaje y aumenta el riesgo de conflictos de nombres a medida que la biblioteca estándar crece.

Tabla 1.2: Comparación de algunas decisiones de diseño del sistema de funciones en distintos lenguajes.

Característica	HULK	Rust	Java	C#	Python	Haskell
Retorno implícito	Sí	Sí	No	No	No	Sí
Sobrecarga de funciones	No	No	Sí	Sí	No	No
Orden libre de declaración	Sí	Sí	Sí	Sí	No	Sí
Funciones matemáticas	Sí (núcleo)	No (<code>std</code>)	No (<code>Math</code>)	No (<code>System.Math</code>)	No (<code>math</code>)	Sí (<code>Prelude</code>)
Organización de funciones	Global	Módulos	Clases	Clases	Módulos	Módulos

1.5. Tipos

Además de los tipos básicos presentados anteriormente, HULK incorpora un sistema de tipos orientado a objetos basado en tipado nominal y herencia simple. La unidad fundamental de este sistema es el **type**, que encapsula un conjunto de atributos y métodos bajo un único identificador. Conceptualmente, esta construcción desempeña el mismo papel que las clases en lenguajes como Java, C++, C# o Python, ya que permite definir nuevos tipos de datos junto con el comportamiento asociado a ellos.

Instanciación de tipos

La creación de objetos en HULK se realiza mediante la expresión **new**, seguida del nombre del tipo y de los valores que se desee pasar a sus parámetros. Un tipo puede declarar argumentos en su propia definición —por ejemplo, **type Point(x, y)**—, que funcionan como un único constructor implícito. Estos argumentos se reciben en el momento de la instanciación y están disponibles dentro de las expresiones de inicialización de los atributos, junto con los símbolos globales del programa. Quedan excluidos de ese entorno tanto la referencia **self** como el resto de atributos de la instancia, de modo que la inicialización de un campo no puede depender del valor de otro campo del mismo objeto. Esta restricción garantiza que las expresiones de inicialización sean independientes entre sí y que el orden en que se evalúen sea irrelevante. Además, evita errores derivados del uso de objetos parcialmente contruidos, un problema que puede aparecer en lenguajes donde el propio constructor puede acceder libremente al estado interno de la instancia.

Este mecanismo de inicialización elimina la posibilidad de definir múltiples constructores con diferentes firmas o incluir lógica arbitraria durante el proceso de construcción del objeto.

El diseño difiere del adoptado por la mayoría de los lenguajes orientados a objetos. En Java, C++ y C#, los constructores constituyen miembros especiales de la clase donde puede ejecutarse lógica arbitraria, realizar validaciones, inicializar recursos o invocar explícitamente al constructor de la superclase mediante mecanismos como **super()** o **base()**.

Python sigue una filosofía similar mediante el método especial `__init__()`. Por el contrario, Rust prescinde de constructores como elemento del lenguaje y suele emplear funciones asociadas, normalmente denominadas `new()`, para encapsular el proceso de creación de los objetos.

Herencia y jerarquía de tipos

HULK organiza todos los tipos definidos por el usuario en una jerarquía de herencia simple cuya raíz es el tipo predefinido `Object`. Todo tipo hereda implícitamente de `Object`, salvo que se especifique otro tipo base mediante la palabra reservada `inherits`.

Esta decisión coincide con la adoptada por lenguajes como Java y C#, que también utilizan herencia simple con una clase raíz común (`Object` u `object`). En cambio, lenguajes como C++ o Python permiten herencia múltiple, posibilitando que un tipo herede simultáneamente de varios padres. Aunque este enfoque proporciona una mayor flexibilidad, también introduce problemas adicionales, como la resolución de conflictos cuando varios ancestros definen un mismo miembro.

Los tipos predefinidos `Number`, `String` y `Boolean` también forman parte de la jerarquía al heredar implícitamente de `Object`. Sin embargo, HULK prohíbe que otros tipos hereden de ellos.

Esta decisión es similar a la adoptada por numerosos lenguajes modernos, donde determinados tipos fundamentales no pueden extenderse o únicamente admiten una herencia muy restringida. El objetivo es preservar las propiedades internas de estos tipos y evitar que una redefinición altere el comportamiento esperado de operaciones básicas del lenguaje.

La inicialización de la jerarquía de herencia también presenta una característica particular en HULK. Si el tipo hijo no declara argumentos propios, hereda automáticamente los del padre, y la instanciación se realiza pasando los valores que el padre espera.

Cuando, por el contrario, se desea que el tipo hijo tenga un conjunto distinto de argumentos, es necesario declararlos explícitamente y, a continuación, indicar cómo se obtienen los argumentos del padre a partir de ellos. Es decir, los argumentos del padre se calculan a partir de los argumentos del hijo. La sintaxis para ello consiste en escribir, junto al nombre del padre, las expresiones de inicialización correspondientes:

```
1         type PolarPoint(phi, rho)
2         inherits Point(rho * sin(phi), rho * cos(phi)) {
3             // ...
4     }
```

De este modo, la relación entre ambos tipos queda completamente especificada en la propia declaración del tipo, sin necesidad de llamadas explícitas al constructor de la superclase durante la construcción del objeto. Esto ofrece un modelo más uniforme pero considerablemente menos expresivo que el proporcionado por los constructores tradicionales.

Encapsulamiento y visibilidad

Una de las decisiones más particulares del sistema de tipos de HULK es que la visibilidad de los miembros no puede ser configurada por el programador. El lenguaje fija una política de encapsulamiento única, eliminando la necesidad de utilizar modificadores de acceso.

Atributos privados

Los atributos de un tipo pueden ser accedidos únicamente desde los métodos definidos en el propio tipo. No es posible leerlos ni modificarlos desde código externo, ni siquiera desde los tipos derivados.

Este comportamiento contrasta con el de lenguajes como Java, C++ o C#, donde el programador puede elegir el nivel de visibilidad de cada atributo (y método) mediante modificadores como `private`, `protected` o `public`. En el extremo opuesto se encuentra Python, donde el lenguaje no incorpora modificadores de acceso, por lo que todos los atributos (y métodos) son accesibles desde el exterior. Aunque existen convenciones de nomenclatura —como el uso del prefijo `_`— e incluso mecanismos como el *name mangling*, ninguno de ellos impone una privacidad real desde el punto de vista del lenguaje.

La decisión adoptada por HULK favorece el encapsulamiento y protege el estado interno de los objetos frente a modificaciones accidentales. Como contrapartida, esta política resulta considerablemente más rígida ya que impide exponer atributos de solo lectura o compartir directamente parte del estado con las clases derivadas mediante mecanismos equivalentes a `protected`.

Métodos públicos y virtuales

Los métodos de un tipo se definen dentro del mismo bloque `type`, con una sintaxis idéntica a la de las funciones globales pero con acceso implícito a la instancia actual mediante el símbolo `self`. A diferencia de Java o C#, donde `this` es una palabra clave, en HULK `self` es un identificador ordinario que el compilador inserta automáticamente en el ámbito del método. Esta decisión tiene la ventaja de mantener la simplicidad del analizador léxico —no introduce nuevas palabras reservadas—, pero implica que `self` puede ser ocultado por un parámetro o una declaración `let` interna. El compilador debe, por tanto, distinguir entre el `self` que refiere a la instancia y cualquier otra variable con el mismo nombre, lo que añade complejidad al análisis semántico. La restricción de que `self` no puede ser destino de una asignación destructiva (`:=`) es una protección sensata que evita la corrupción accidental de la referencia a la instancia.

A diferencia de los atributos, todos los métodos definidos en un tipo forman parte automáticamente de su interfaz pública y pueden ser invocados desde cualquier punto del programa. Asimismo, todos ellos son virtuales por definición.

Desde la perspectiva del diseño, esta decisión reduce el número de conceptos que el programador debe manejar y favorece un modelo uniforme para todos los tipos. Sin embargo, elimina la posibilidad de definir métodos exclusivamente internos o restringir el acceso a determinadas operaciones cuando ello resulte conveniente.

Polimorfismo

Todos los métodos de HULK participan automáticamente en el mecanismo de despacho dinámico, por lo que cualquier método puede ser redefinido por un tipo derivado siempre que conserve exactamente la misma firma. Para acceder a la implementación heredada del ancestro más próximo, el lenguaje proporciona el identificador especial `base`.

Este enfoque se aproxima al de Java, donde la mayoría de los métodos son virtuales por defecto, aunque existen excepciones como los métodos `static`, `private` o `final`. En C++ y C#, por el contrario, el comportamiento virtual debe indicarse explícitamente mediante la palabra clave `virtual`, mientras que la redefinición suele acompañarse de

override. El identificador **base** desempeña un papel equivalente al de **super** en Java o al propio **base** en C#, permitiendo reutilizar la implementación heredada.

Considerar virtuales todos los métodos favorece la extensibilidad y elimina una decisión adicional para el programador, ya que cualquier operación puede ser redefinida automáticamente en una jerarquía de herencia. Como contrapartida, el desarrollador pierde la posibilidad de impedir que determinados métodos sean sobrescritos, un mecanismo utilizado en otros lenguajes para preservar invariantes de implementación o controlar con mayor precisión la evolución de una jerarquía de tipos.

1.6. Sistema de tipos

1.6.1. Tipado estático e inferencia de tipos

HULK adopta un sistema de tipos estático con anotaciones opcionales. Todas las expresiones y símbolos del programa poseen un tipo conocido durante la compilación, aunque el programador no está obligado a anotar explícitamente dichos tipos en la mayoría de los casos.

Esta decisión sitúa a HULK en una posición intermedia entre los lenguajes de tipado dinámico, como Python, y aquellos que requieren anotaciones explícitas para la mayoría de las declaraciones, como Java o C++. El objetivo es combinar las garantías del tipado estático con una sintaxis menos verbosa, permitiendo que el compilador deduzca automáticamente los tipos cuando dispone de información suficiente.

La inferencia de tipos constituye la primera fase del análisis semántico. Durante esta etapa el compilador intenta determinar el tipo de todas las expresiones y de aquellos símbolos cuya anotación ha sido omitida. Una vez finalizada la inferencia, el comprobador de tipos verifica que todas las operaciones sean compatibles con los tipos inferidos y con las anotaciones explícitas presentes en el programa.

A diferencia de lenguajes como Haskell, cuya especificación define un algoritmo de inferencia muy preciso, HULK únicamente establece las propiedades que debe satisfacer cualquier mecanismo de inferencia correcto. En consecuencia, distintas implementaciones del lenguaje pueden diferir en los casos en que son capaces de inferir automáticamente un tipo, siempre que respeten las restricciones impuestas por la especificación.

Cuando el compilador no puede determinar un tipo de forma inequívoca, el programador debe proporcionarlo mediante una anotación explícita. Es decir, si tras la fase de inferencia permanecen tipos indeterminados, el proceso de comprobación de tipos informa un error de compilación. De este modo, la inferencia actúa como un mecanismo de comodidad para el programador, sin comprometer las garantías del tipado estático.

Como contrapartida, HULK no incorpora polimorfismo paramétrico mediante tipos genéricos, presente en lenguajes como Java, C#, Rust o Kotlin. La ausencia de este mecanismo simplifica tanto la inferencia como el sistema de tipos, aunque reduce la capacidad de escribir componentes reutilizables independientes del tipo concreto con el que operan.

1.6.2. Conformidad de tipos

Una vez establecidos los tipos de todas las expresiones, ya sea mediante inferencia o mediante anotaciones explícitas, el compilador debe comprobar que dichos tipos son compatibles en cada contexto donde se espera uno concreto: al pasar argumentos a una

función, al asignar un valor a una variable o al retornar desde una función. A esta relación de compatibilidad se la denomina *conformidad de tipos*.

En HULK, la conformidad sigue un modelo nominal. Un tipo T conforma a un tipo S si T es exactamente S , o bien si T hereda directa o indirectamente de S . La jerarquía de tipos tiene como raíz a `Object`, del que derivan implícitamente todos los tipos que no declaran un ancestro explícito. Por tanto, cualquier tipo conforma a `Object`. Además, los tipos básicos (`Number`, `String` y `Boolean`) son tipos finales: los únicos tipos que conforman con ellos son ellos mismos.

Este mecanismo coincide con el utilizado por el sistema de clases de Java y C#, donde la conformidad entre clases también viene determinada por la relación de herencia. No obstante, estos lenguajes incorporan un segundo mecanismo de conformidad mediante interfaces, que permite que un tipo conforme a una interfaz incluso sin heredar de ella. En HULK base no existen las interfaces ni los protocolos, por lo que la conformidad se basa exclusivamente en la relación de herencia. Esto simplifica el sistema de tipos, pero también lo hace más rígido.

La ventaja de este enfoque nominal es su simplicidad: el compilador solo necesita recorrer la jerarquía de herencia para decidir si una asignación o una invocación es válida, y el programador puede razonar sobre la compatibilidad de tipos con reglas claras y predecibles. Además, la conformidad permite calcular el *ancestro común más cercano* (*lowest common ancestor*, LCA). Este mecanismo resulta especialmente relevante en HULK debido a que los condicionales son expresiones y, por tanto, siempre deben poseer un tipo de retorno.

La principal limitación del modelo nominal es su rigidez. Dos tipos que comparten exactamente los mismos métodos y atributos, pero que no están relacionados por herencia, son completamente incompatibles desde el punto de vista del compilador. Esto impide formas de polimorfismo que en otros lenguajes se consiguen mediante mecanismos alternativos:

- En lenguajes de tipado estructural como TypeScript o Go, la compatibilidad se determina por la presencia de los métodos o campos requeridos, independientemente del nombre del tipo o de su posición en una jerarquía. En Go, además, esta conformidad es implícita: el tipo no necesita declarar que implementa una interfaz; basta con que proporcione los métodos necesarios.
- En lenguajes con *traits* (Rust) o *type classes* (Haskell), la compatibilidad se basa en la implementación de un conjunto de operaciones, lo que permite que tipos no relacionados compartan comportamientos sin necesidad de herencia. En estos mecanismos, la reutilización se fundamenta en las capacidades que ofrece un tipo y no en su posición dentro de una jerarquía de clases.

Estos mecanismos permiten expresar formas de polimorfismo más flexibles que la herencia nominal, favoreciendo la reutilización de componentes entre tipos que no comparten una relación de herencia directa.

1.6.3. Comprobación y conversión dinámica de tipos

La conformidad nominal permite que una variable se declare con el tipo de una clase base y almacene instancias de cualquiera de sus descendientes. Sin embargo, esta flexibilidad introduce una situación en la que el tipo estático conocido por el compilador deja de coincidir necesariamente con el tipo dinámico del objeto almacenado durante la ejecución.

Para resolver esta situación, HULK incorpora dos operadores complementarios: `is`, que permite comprobar en tiempo de ejecución si el tipo dinámico de una expresión conforma a un tipo determinado, y `as`, que realiza una conversión descendente (*downcasting*) hacia dicho tipo. Mientras que `is` produce simplemente un valor booleano, `as` modifica el tipo estático con el que el compilador interpreta la expresión. Si el objeto no pertenece realmente al tipo solicitado, la conversión produce un error en tiempo de ejecución.

Este diseño sigue el modelo clásico de lenguajes orientados a objetos como Java (`instanceof` y `cast`) o C# (`is` y `as`), donde el programador puede recuperar información sobre el tipo dinámico cuando la jerarquía de herencia hace insuficiente el conocimiento estático del compilador. Frente a otros lenguajes como C++, HULK ofrece una sintaxis más simple y específica para esta operación.

La principal ventaja de este mecanismo es que permite aprovechar el polimorfismo basado en herencia sin renunciar a acceder a las operaciones específicas de un subtipo cuando resulta necesario. Además, la separación entre la comprobación (`is`) y la conversión (`as`) hace explícita la intención del programador y evita conversiones implícitas potencialmente ambiguas.

Como contrapartida, el operador `as` introduce un punto en el que la seguridad del programa deja de depender exclusivamente del comprobador de tipos y pasa a depender también del comportamiento en tiempo de ejecución. Un uso incorrecto puede provocar un error dinámico, por lo que la responsabilidad de garantizar la validez de la conversión recae en el programador.

Lenguajes más recientes han evolucionado hacia mecanismos que integran ambas operaciones. Kotlin incorpora *smart casts*, de forma que tras una comprobación con `is` el compilador refina automáticamente el tipo de la variable, eliminando la necesidad de un `as` explícito. De manera similar, Rust emplea *pattern matching* para combinar la comprobación y el acceso al valor en una única construcción segura. Estas aproximaciones reducen la verbosidad y evitan que el programador pueda realizar conversiones potencialmente inseguras.

Capítulo 2

Extensiones añadidas al lenguaje

Una vez definido el núcleo del lenguaje es posible analizar una serie de construcciones adicionales que amplían su expresividad sin modificar sus principios fundamentales.

El objetivo de este capítulo es describir las extensiones implementadas por el equipo, su sintaxis, semántica e integración con el compilador.

2.1. Polimorfismo paramétrico en funciones

Motivación

En la especificación original, HULK no incorpora polimorfismo paramétrico. Como consecuencia, cada función posee un único tipo estático y no puede reutilizarse con argumentos de distinta naturaleza, incluso cuando su implementación es completamente independiente del tipo de los datos que manipula.

Considérese la función identidad:

```
1      function id(x) => x;
```

Conceptualmente, esta función debería poder recibir un valor de cualquier tipo y devolver exactamente ese mismo valor. Sin embargo, al no existir parámetros de tipo, la función debe asociarse a un único tipo concreto durante la compilación. Esto impide reutilizar una misma implementación con distintos tipos de argumentos, por lo que el programador se ve obligado a escribir múltiples versiones equivalentes de la misma función cuando desea reutilizarla con distintos tipos de datos.

Por ejemplo, el siguiente programa expresa una intención natural desde el punto de vista del programador:

```
1      print(id(42));  
2      print(id("Hola"));
```

No obstante, ambas llamadas no pueden coexistir sobre una única definición de `id`.

Esta restricción reduce la capacidad de abstracción del lenguaje y dificulta la construcción de bibliotecas reutilizables.

Solución

Para superar esta limitación se incorporó soporte para funciones genéricas, permitiendo que una misma definición pueda utilizarse con distintos tipos de argumentos sin necesidad de duplicar su implementación.

La genericidad puede declararse explícitamente mediante el parámetro de tipo reservado `T`:

```
1         function id(x: T): T => x;
```

Para soportar esta sintaxis fue necesario extender la gramática del lenguaje, permitiendo que las anotaciones de tipo acepten, además de los tipos concretos (`Number`, `String`, `Boolean` y los definidos por el usuario), el identificador reservado `T`.

No obstante, la anotación explícita es opcional, el programador puede omitir completamente las anotaciones:

```
1         function id(x) => x;
```

Ambas definiciones son equivalentes, por lo que el programa presentado en la motivación pasa a ser válido:

```
1         print(id(42));
2         print(id("Hola"));
3         print(id(true));
```

La extensión mantiene la filosofía de diseño del lenguaje. Del mismo modo que HULK permite omitir las anotaciones de tipo en variables, funciones o atributos, la genericidad también puede expresarse de forma implícita o explícita, dejando al programador decidir cuándo desea documentar la interfaz de una función y cuándo prefiere confiar en la inferencia del compilador.

Asimismo, la incorporación de funciones genéricas no modifica el comportamiento de los programas existentes, sino que amplía el conjunto de programas aceptados. Las funciones cuyo cuerpo restringe el tipo de sus parámetros continúan siendo monomórficas, mientras que aquellas cuya implementación es independiente del tipo pueden reutilizarse con distintas instanciaciones.

Conviene destacar que una función genérica no puede utilizarse indiscriminadamente con cualquier tipo. Aunque la función sea genérica, los tipos con los que puede instanciarse siguen estando determinados por las operaciones que realiza su implementación. Si un determinado tipo no satisface dichas operaciones, el compilador detectará la incompatibilidad durante la comprobación de tipos y rechazará la llamada antes de la ejecución del programa.

Esta extensión sitúa a HULK más cerca de lenguajes como Java, C#, C++ o Rust, todos ellos con soporte para polimorfismo paramétrico. Sin embargo, el enfoque adoptado difiere en un aspecto importante: mientras estos lenguajes exigen que la genericidad se declare explícitamente en la interfaz de la función, HULK mantiene dicha anotación como opcional.

2.2. Protocolos

Motivación

El sistema de tipos original de HULK es puramente nominal: dos tipos solo son compatibles si pertenecen a la misma jerarquía de herencia. Como consecuencia, dos clases que ofrecen exactamente el mismo comportamiento siguen siendo incompatibles si no comparten un ancestro común.

Esta restricción dificulta la reutilización de código y obliga a diseñar las jerarquías de clases con antelación, incluso cuando únicamente se desea expresar que distintos tipos proporcionan un mismo conjunto de operaciones.

Solución

Para superar esta limitación se incorporó soporte para protocolos. Los protocolos incorporan al lenguaje un mecanismo de abstracción basado en tipado estructural, complementando el sistema nominal de clases ya existente. Un tipo conforma a un protocolo cuando proporciona los métodos exigidos por este, sin necesidad de declarar explícitamente que lo implementa.

Este modelo es muy similar al empleado por Go y, en cierta medida, por TypeScript, donde la conformidad también se determina implícitamente a partir de la estructura del tipo. En cambio, lenguajes como Java, C#, Rust o Swift requieren que la implementación de interfaces o *traits* se declare de forma explícita, lo que hace más visible la intención del programador.

La principal ventaja del enfoque adoptado en HULK es que favorece la reutilización y el desacoplamiento. Un protocolo puede definirse independientemente de los tipos existentes, y cualquier tipo que proporcione los métodos adecuados conformará automáticamente, incluso aunque haya sido desarrollado con anterioridad. Además, los protocolos pueden extender otros protocolos y admiten covarianza en los tipos de retorno y contravarianza en los parámetros, lo que incrementa su expresividad sin alterar el modelo de herencia de clases.

Como contrapartida, la implementación implícita hace menos evidente qué protocolos satisface un tipo únicamente observando su declaración. Mientras que en lenguajes con implementación explícita esta información forma parte de la interfaz pública, en HULK debe deducirse a partir de los métodos definidos por el tipo.

2.3. Iterables

Motivación

En el diseño original de HULK, el bucle `for` estaba restringido a la función predefinida `range`, lo que limitaba su utilidad a la iteración sobre secuencias numéricas. Esta decisión simplificaba la implementación inicial, pero suponía una seria limitación desde el punto de vista de la expresividad del lenguaje. En lenguajes modernos, el bucle `for` se concibe como una construcción universal capaz de recorrer cualquier tipo de datos que exponga una interfaz de iteración.

Sin un mecanismo abstracto de iteración, el desarrollador se ve forzado a escribir bucles `while` manuales para cada tipo de colección, acoplando la lógica de control a la estructura concreta y dificultando la reutilización de código. Además, se pierde la oportunidad de definir iteraciones perezosas o personalizadas —por ejemplo, recorridos con filtros o transformaciones— que son comunes en el desarrollo moderno.

La extensión de iterables aborda estas carencias, permitiendo que cualquier tipo definido por el usuario pueda ser utilizado en un bucle `for`, siempre que implemente un conjunto mínimo de operaciones. De este modo, HULK se alinea con la filosofía de lenguajes que priorizan la composición y la abstracción, como Python, Rust o Java.

Solución

Para dotar al lenguaje de un mecanismo de iteración uniforme, se definieron dos protocolos fundamentales, inyectados como parte del núcleo del lenguaje:

```

1 protocol Iterable {
2     next(): Boolean; // Avanza al siguiente elemento; devuelve
        true si hay elemento
3     current(): Object; // Devuelve el elemento actual
4 }

```

El protocolo `Iterable` establece el contrato mínimo para un recorrido de una sola pasada. Cualquier tipo que proporcione estos métodos con las firmas adecuadas conforma automáticamente a `Iterable`, gracias al sistema de protocolos de HULK (tipado estructural).

Sin embargo, este protocolo solo permite una única iteración: una vez que el iterador ha llegado al final, no puede reiniciarse. Para aquellos tipos que necesitan ser recorridos múltiples veces (por ejemplo, una lista o un rango reutilizable), se definió un segundo protocolo:

```

1 protocol Enumerable {
2     iter(): Iterable; // Devuelve un nuevo iterador para la
        coleccion
3 }

```

Un tipo que implementa `Enumerable` garantiza que cada invocación a `iter()` devuelve un iterador independiente y en su estado inicial, permitiendo bucles anidados o múltiples recorridos sobre los mismos datos sin interferencias. Es importante destacar que el programador *no* necesita llamar explícitamente a `iter()` al utilizar un objeto `Enumerable` en un bucle `for`. El compilador realiza esta transformación de forma automática durante la generación de código: dado un bucle `for (x in enumerable) cuerpo`, el backend lo reescribe como una combinación de `let` y `while` que invoca `iter()` para obtener el iterador y luego usa `next()` y `current()` para recorrerlo. De este modo, el código fuente permanece limpio y el usuario solo se preocupa por la semántica de la iteración, no por los detalles de implementación.

Soporte para `range` como iterable predefinido

La función predefinida `range(inicio, fin)` devuelve una instancia del tipo interno `_Range`, que se inyecta durante el análisis semántico. Este tipo posee los atributos `min`, `max` y `current`, e implementa el protocolo `Iterable` mediante los métodos:

```

1 next(): Boolean => (self.current := self.current + 1) < self.max;
2 current(): Number => self.current;

```

De este modo, `range` se integra perfectamente en el mecanismo de iteración, sin necesidad de un tratamiento especial en el backend más allá de la creación del objeto. Además, al implementar `Iterable` y no `Enumerable`, cada iteración sobre un rango produce un nuevo iterador implícitamente, lo que permite múltiples usos de `range` en distintos bucles.

Tipado de iterables con `T*`

Dado que el protocolo `Iterable` define `current(): Object`, anotar funciones o variables con `Iterable` resultaría incómodo, ya que obligaría al programador a realizar conversiones descendentes (*downcasting*) para acceder a los elementos con su tipo concreto. Para resolver este problema, HULK ofrece una sintaxis especial: `T*` para denotar un iterable cuyos elementos son de tipo `T`.

Por ejemplo, la siguiente función suma los elementos de un iterable de números:

```

1      function sum(numbers: Number*): Number =>
2      let total = 0 in
3      for (x in numbers)
4      total := total + x;

```

El compilador interpreta `Number*` como `Iterable(Number)`. A nivel de implementación interna, cuando se utiliza `T*` en cualquier parte del programa, el compilador inserta automáticamente un protocolo especializado con la siguiente estructura:

```

1      protocol Iterable_T extends Iterable {
2          current(): T;
3      }

```

Este protocolo redefine el método `current()` para que devuelva el tipo concreto `T` en lugar de `Object`, respetando las reglas de covarianza en los tipos de retorno. De este modo, el compilador puede verificar que el uso de los elementos en el bucle es tipo-seguro sin necesidad de conversiones explícitas.

Esta extensión convierte al bucle `for` en una construcción verdaderamente universal, capaz de iterar sobre cualquier tipo que exponga el comportamiento adecuado, sentando las bases para futuras abstracciones (como funciones de orden superior sobre iterables) sin modificar el núcleo del lenguaje..

Capítulo 3

Pipeline de compilación

En los capítulos anteriores se describieron las características del lenguaje HULK y las extensiones incorporadas durante el desarrollo del proyecto. El compilador que le da soporte, implementado en Rust, se organiza en cadena de etapas sucesivas. Cada etapa consume la representación producida por la etapa anterior y genera una nueva representación que acerca el programa a su forma final ejecutable. Esta organización permite desarrollar, probar y depurar cada fase de forma independiente, y facilita la incorporación de nuevas etapas cuando el lenguaje se extiende.

La entrada del compilador es un archivo fuente con extensión `.hulk`. La salida final es un archivo `output.ll` que contiene la representación intermedia LLVM (LLVM IR) del programa. Este archivo puede ser posteriormente compilado a código nativo mediante `clang` y ejecutado, o bien procesado por otras herramientas del ecosistema LLVM.

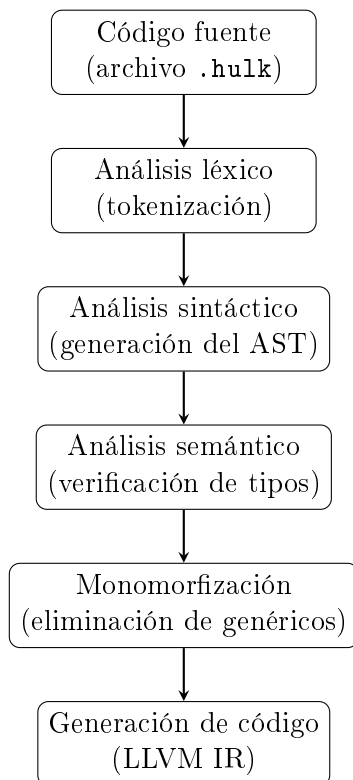


Figura 3.1: Pipeline de compilación de HULK.

3.1. Análisis léxico

El análisis léxico constituye la primera etapa del pipeline de compilación y tiene como objetivo transformar el código fuente en una secuencia de tokens que será consumida posteriormente por el analizador sintáctico. Su implementación sigue la arquitectura clásica basada en expresiones regulares y autómatas finitos descrita en *Compilers: Principles, Techniques, and Tools*. No obstante, el analizador desarrollado combina la simulación de un autómata determinista con código especializado para aquellos lexemas cuya descripción resulta más sencilla mediante procedimientos imperativos, como los números, las cadenas de texto o los identificadores. Esta aproximación híbrida mantiene los fundamentos teóricos de los analizadores léxicos clásicos y, al mismo tiempo, simplifica la implementación y facilita su extensión.

Especificación de los tokens

Los elementos léxicos del lenguaje se representan mediante el tipo enumerado `Token`, que agrupa palabras reservadas, operadores, símbolos de puntuación, literales, identificadores y las nuevas construcciones añadidas durante la extensión del lenguaje.

Los operadores y símbolos fijos se describen mediante expresiones regulares literales, mientras que las palabras reservadas se distinguen posteriormente a partir del lexema reconocido. Esta organización mantiene una representación uniforme de todos los tokens y facilita la incorporación de nuevos elementos al lenguaje sin modificar la arquitectura general del analizador.

Construcción del autómata

Cada expresión regular es procesada inicialmente por un analizador que construye su árbol sintáctico, representando los operadores fundamentales de las expresiones regulares: concatenación, unión, clausura de Kleene y símbolos literales.

A partir de este árbol se aplica el algoritmo de Thompson para construir un autómata finito no determinista (NFA) con un único estado inicial y un único estado final. Posteriormente, los autómatas correspondientes a todos los patrones se unifican en un único NFA mediante un nuevo estado inicial conectado por transiciones ϵ .

Finalmente, el autómata resultante se transforma en un autómata finito determinista (DFA) utilizando la construcción de subconjuntos. Cada estado del DFA representa un conjunto de estados del NFA y almacena, cuando corresponde, el token asociado al estado de aceptación de mayor prioridad. Esta conversión permite realizar el reconocimiento de tokens mediante una simulación completamente determinista.

Tokenización de la entrada

Durante la compilación el código fuente se recorre secuencialmente simulando el DFA construido previamente. Desde cada posición de la entrada el analizador avanza mientras exista una transición válida, registrando el último estado de aceptación alcanzado. Cuando ya no es posible continuar, se genera el token correspondiente al prefijo reconocido más largo y el proceso continúa desde la posición siguiente.

Este procedimiento implementa la estrategia de *longest match*, resolviendo correctamente conflictos entre operadores que comparten prefijos, como `=` y `==`, `:` y `:=`, o `@` y `@@`.

Cuando varios patrones reconocen un mismo lexema con la misma longitud, se respeta la prioridad establecida durante la construcción del autómata.

Reconocimiento manual de literales

Aunque el DFA reconoce la mayoría de los tokens del lenguaje, determinados lexemas requieren un tratamiento específico que resulta más sencillo mediante código procedural que mediante expresiones regulares.

Por este motivo, antes de iniciar la simulación del autómata, el lexer detecta de forma secuencial varios casos particulares:

1. espacios en blanco y comentarios, que se descartan;
2. literales numéricos enteros y reales;
3. cadenas de caracteres, incluyendo el procesamiento de secuencias de escape;
4. identificadores y palabras reservadas, distinguiendo posteriormente si el lexema corresponde a una palabra clave o a un identificador definido por el usuario.

Esta estrategia reduce la complejidad de las expresiones regulares necesarias y permite un tratamiento más preciso de aquellos elementos léxicos que requieren comprobaciones adicionales durante su reconocimiento.

3.2. Análisis sintáctico

El analizador sintáctico constituye la segunda etapa del compilador y tiene como objetivo verificar que la secuencia de tokens producida por el analizador léxico cumple la gramática del lenguaje. Cuando la entrada es válida, construye el árbol de sintaxis abstracta (AST), representación intermedia que elimina detalles puramente sintácticos y preserva únicamente la estructura necesaria para las fases posteriores del compilador.

La sintaxis de HULK se especifica mediante una gramática libre de contexto, donde cada producción describe la forma en que pueden combinarse los distintos elementos del lenguaje para formar construcciones válidas. A partir de dicha gramática se genera automáticamente el parser utilizando la herramienta LALRPOP.

Fundamentos teóricos

Los analizadores sintácticos pueden clasificarse en dos grandes familias: descendentes (*LL*) y ascendentes (*LR*).

Los analizadores *LL* construyen el árbol sintáctico desde la raíz hacia las hojas, expandiendo sucesivamente los no terminales mediante una derivación por la izquierda. Su principal ventaja es la simplicidad de implementación; sin embargo, requieren gramáticas especialmente adaptadas, eliminando la recursividad por la izquierda y realizando factorizaciones cuando existen prefijos comunes.

Por el contrario, los analizadores *LR* construyen el árbol en sentido inverso, partiendo de los terminales reconocidos y reduciéndolos progresivamente hasta obtener el símbolo inicial de la gramática. Este enfoque permite reconocer una familia considerablemente

más amplia de gramáticas y manejar de forma natural reglas recursivas por la izquierda, muy frecuentes en la definición de expresiones aritméticas y operadores.

Dentro de la familia LR existen varias variantes:

- **SLR (Simple LR)**
- **LR(1) canónico**
- **LALR(1) (Look-Ahead LR)**

Elección de LALRPOP

El compilador se implementó utilizando LALRPOP, un generador de analizadores sintácticos para Rust basado en el algoritmo LALR(1). Esta elección responde tanto a criterios teóricos como prácticos:

- **Manejo natural de la recursividad izquierda:** HULK tiene una amplia variedad de operadores con distintas precedencias y asociatividades. Con un analizador LL, la gramática de expresiones tendría que reescribirse extensamente, haciéndola menos legible y más propensa a errores. LALR(1) maneja reglas como `AddSubExpr → AddSubExpr + MulDivExpr` sin problemas, respetando la asociatividad izquierda de forma directa.
- **Potencia expresiva:** LALR(1) cubre prácticamente todas las construcciones sintácticas de los lenguajes de programación modernos (clases, herencia, protocolos, macros, bucles). Es el “punto óptimo” entre la potencia del LR(1) canónico y la compacidad del SLR.
- **Herramienta adecuada (LALRPOP):** LALRPOP está escrito en Rust y se integra perfectamente con el ecosistema del proyecto. Genera código limpio y seguro, con excelentes mensajes de error que facilitan el desarrollo iterativo. Permite incrustar acciones semánticas (construcción del AST) directamente en las reglas gramaticales.
- **Escalabilidad:** HULK es un lenguaje incremental que añade características complejas. Un analizador LR escala mejor a medida que la gramática crece, ya que no requiere reestructuraciones profundas para acomodar nuevas reglas, a diferencia de los analizadores LL que necesitan factorización izquierda constante.

Implementación del parser

La gramática del lenguaje se encuentra definida en el archivo `grammar.lalrpop`, que contiene todas las producciones necesarias para reconocer la sintaxis de HULK. La gramática organiza las expresiones en una jerarquía de reglas que refleja fielmente la precedencia de los operadores, desde los de menor prioridad hasta los de mayor prioridad. Por ejemplo, la regla `LogicalOrExpr` reconoce la disyunción lógica y se compone de expresiones de tipo `LogicalAndExpr`; a su vez, `LogicalAndExpr` está formada por `ComparisonExpr`, y así sucesivamente hasta llegar a `AtomicExpr`. Esta estructura evita la necesidad de una tabla de precedencia externa y permite que el propio parser construya el árbol respetando el orden de evaluación esperado.

Las reglas auxiliares `CommaSep<T>` y `Comma<T>` se utilizan para manejar listas de elementos separados por comas, lo que simplifica la definición de parámetros, argumentos y

bindings de `let`. Gracias a estas reglas, el parser puede recolectar secuencias de elementos de forma uniforme y construir vectores que se almacenan directamente en los nodos del AST correspondientes.

El reconocimiento de tipos está integrado en la gramática mediante la regla **Type**, que distingue entre tipos básicos (**Number**, **String**, **Boolean**), tipos definidos por el usuario (**identifier**), el marcador genérico **T**, y tipos iterables con el sufijo ***** (por ejemplo, **Number***). Esta representación se utiliza en las anotaciones de variables, parámetros, atributos, retornos de funciones y métodos, así como en los argumentos de tipo de las definiciones de clases. Durante la construcción del AST, las anotaciones de tipo se almacenan como **Option<HulkType>**, y el marcador **T** se filtra para ser manejado posteriormente en el análisis semántico, permitiendo que la genericidad se exprese de forma opcional.

El parser también distingue entre **SimpleExpr** —que abarca todas las expresiones que no son bloques— y **BlockExpr**, que permite secuencias de expresiones entre llaves. La regla **BlockInner** recolecta las expresiones separadas por punto y coma, y toma la última como valor del bloque, respetando la semántica de HULK. De esta forma, el AST refleja directamente la estructura del código fuente, facilitando el recorrido y las transformaciones posteriores.

Dentro de la implementación de la gramática, una de las decisiones de diseño más relevante es la desugarización de las cláusulas `elif`, explicada más a detalle a continuación.

Desugarización del `elif`

El lenguaje HULK incorpora la cláusula `elif` para facilitar condicionales múltiples. En lugar de crear un nodo específico en el AST para `elif`, el parser convierte todas las ramas en una cadena de expresiones `if-else` anidadas. Por ejemplo, la construcción:

```
if (cond1) body1
elif (cond2) body2
elif (cond3) body3
else body4
```

se transforma en el siguiente AST anidado:

```
IfExpr {
  condition: cond1,
  then_branch: body1,
  else_branch: IfExpr {
    condition: cond2,
    then_branch: body2,
    else_branch: IfExpr {
      condition: cond3,
      then_branch: body3,
      else_branch: body4
    }
  }
}
```

Para construir esta estructura, el parser recolecta todas las ramas `elif` y, partiendo desde la rama `else` final, va envolviendo cada `elif` en un nodo **IfExpr** estándar, de

modo que la primera condición queda en el nivel más externo y las condiciones siguientes se evalúan solo si las anteriores son falsas. Esta decisión mantiene el AST con un único tipo de nodo condicional, lo que simplifica el visitante que realiza el análisis semántico y la generación de código, a la vez que preserva la semántica de evaluación en cortocircuito, ya que la estructura anidada refleja fielmente la lógica original.

3.3. Árbol de Sintaxis Abstracta

El Árbol de Sintaxis Abstracta (AST) constituye la representación interna del programa. A diferencia del árbol sintáctico derivado directamente de la gramática, el AST elimina los elementos puramente sintácticos (como separadores o paréntesis) y conserva únicamente la información necesaria para representar la estructura semántica del programa.

Estructura general del árbol

La raíz del árbol está representada por el nodo **Program**, encargado de agrupar las distintas categorías de definiciones del lenguaje. En particular, almacena las definiciones de tipos, protocolos y funciones, además de la expresión principal que constituye el punto de entrada del programa.

Esta organización refleja directamente la estructura global de HULK y permite que las distintas fases del compilador procesen de forma independiente cada clase de declaración cuando resulta necesario.

Representación de expresiones

Todas las construcciones evaluables del lenguaje se representan mediante el enumerado **Expr**. Cada variante del enumerado corresponde a una forma de expresión distinta, como operaciones binarias, llamadas a funciones, condicionales, bloques, bucles, creación de objetos o accesos a atributos.

Esta decisión resulta coherente con el propio diseño de HULK, ya que el lenguaje es orientado a expresiones: construcciones como **if**, **while** o los bloques producen un valor y pueden aparecer allí donde se espera cualquier otra expresión.

Representación de declaraciones

Las distintas declaraciones del lenguaje se representan mediante nodos especializados, entre los que destacan **FunctionDef**, **TypeDef** y **ProtocolDef**. Cada uno encapsula únicamente la información propia de la entidad que representa.

Las funciones almacenan su nombre, parámetros, tipo de retorno y cuerpo; los tipos incluyen su posible relación de herencia, atributos y métodos; mientras que los protocolos contienen únicamente las firmas de los métodos que definen su interfaz. Esta separación evita estructuras excesivamente generales y simplifica el procesamiento posterior de cada clase de declaración.

Información semántica asociada

Además de la información sintáctica, todos los nodos del AST incorporan datos que serán utilizados durante las fases posteriores del compilador. En particular, cada nodo almacena su posición en el código fuente mediante un `Span`, lo que permite producir mensajes de error precisos indicando la línea y la columna donde se detectó cada problema.

En aquellos elementos donde el lenguaje admite anotaciones de tipo (parámetros, atributos o tipos de retorno), el AST distingue entre el tipo especificado por el programador (`ty_annotation`) y el tipo finalmente asignado por el compilador (`ty`). Esta separación permite preservar la información original del programa durante todo el proceso de compilación y, al mismo tiempo, registrar el resultado de la inferencia y comprobación de tipos. Gracias a ello, el análisis semántico puede verificar la consistencia entre las anotaciones explícitas y los tipos inferidos sin perder ninguna de las dos representaciones.

Patrón Visitor

Todos los nodos del AST implementan la interfaz `Node`, que define el método `accept` correspondiente al patrón de diseño *Visitor*. Cada fase del compilador implementa su propio visitante para recorrer el árbol y realizar una tarea concreta, como el análisis semántico o la generación de código LLVM.

Este enfoque desacopla la estructura del AST de las operaciones realizadas sobre él. Como consecuencia, es posible incorporar nuevas fases al compilador sin modificar la definición de los nodos, manteniendo una arquitectura modular y facilitando la evolución del proyecto.

3.4. Análisis semántico

El análisis semántico es la fase encargada de verificar la corrección lógica del programa y de enriquecer el AST con la información de tipos necesaria para las etapas posteriores. Esta tarea recae sobre el `TypeChecker`, que recorre el árbol mediante el patrón Visitor apoyándose en tres pilares fundamentales: un sistema de tipos nominal con inferencia, un motor de unificación basado en restricciones y un proceso de aplanamiento que transforma la jerarquía de clases en una representación plana lista para el backend.

Inferencia y comprobación de tipos

El sistema de tipos se modela mediante el enumerado `HulkType`, que representa tanto los tipos concretos del lenguaje (`Number`, `String`, `Boolean`, `Object` y las clases definidas por el usuario) como construcciones auxiliares necesarias para la inferencia. Entre estas últimas destacan las **variables de tipo** (`Var`), que actúan como marcadores para tipos aún desconocidos, y el tipo absorbente `Error`, que propaga fallos sin generar nuevos diagnósticos en expresiones ya inválidas.

El motor de **unificación** (`Unifier`) es el corazón del proceso de inferencia. Mantiene una tabla de sustituciones que asocia variables de tipo con los tipos concretos a los que han sido ligadas, y una tabla de restricciones que limitan la posible resolución de cada variable. Durante el recorrido del AST, cada operación impone ecuaciones sobre los tipos de sus operandos. El método `unify` resuelve estas ecuaciones: si uno de los tipos es una variable, intenta ligarla al otro mediante `bind`; si ambos son concretos, verifica que sean

compatibles estructuralmente. La operación `bind` incluye dos salvaguardas esenciales: el *occurs check*, que impide crear ciclos infinitos al ligar una variable a un tipo que la contiene, y la verificación de restricciones asociadas a la variable, que asegura que el tipo concreto cumpla las condiciones semánticas exigidas.

Actualmente se soportan dos restricciones semánticas. La restricción `StringOrNumber`, impuesta por los operadores de concatenación `@` y `@@`, obliga a que la variable se ligue exclusivamente a `String` o `Number`, ya que ambos son convertibles a cadena. La restricción `ConformsToProtocol`, asociada a parámetros anotados con un protocolo, exige que cualquier tipo que se una a la variable implemente estructuralmente todos los métodos del protocolo, respetando las reglas de varianza.

Este mecanismo permite que el `TypeChecker` deduzca los tipos de la mayoría de las expresiones sin anotaciones explícitas. Cuando no es posible inferir el tipo de una variable, el compilador reporta un error y solicita al programador que proporcione una anotación explícita. Sin embargo, cuando la falta de información afecta a los parámetros o retorno de una función definida por el usuario, el compilador adopta una estrategia distinta: en lugar de rechazar el programa, la variable correspondiente permanece sin ligar y la función se marca como genérica. La resolución de estas funciones genéricas se pospone a una fase posterior del compilador, la monomorfización, que se describe en la sección 3.5.

Aplanamiento de la jerarquía

Antes de que el backend pueda generar código, la jerarquía de clases debe transformarse en una representación plana que elimine toda dependencia de la herencia. Esta tarea recae sobre el proceso de **aplanamiento**, que para cada tipo definido por el usuario construye una estructura `FlattenedType` con tres componentes esenciales: una lista ordenada de todos los atributos que un objeto de esa clase tendrá en memoria (comenzando por los heredados y continuando con los propios), un vector de métodos efectivos con sus índices para la tabla virtual (VTable), y la lista de parámetros que realmente acepta el constructor de la clase.

Durante este proceso se resuelven las relaciones de herencia. Si un hijo sobrescribe un método del padre, la nueva implementación ocupa exactamente la misma posición en la VTable, garantizando la compatibilidad con el polimorfismo. Los métodos nuevos reciben índices incrementales al final de la tabla. De manera análoga, si el hijo proporciona sus propios parámetros y expresiones de inicialización para el constructor padre, estas se evalúan y unifican en este punto, asegurando que los tipos de los argumentos coincidan con los esperados.

El resultado es una representación completamente libre de referencias a la herencia, donde el backend puede acceder a atributos y métodos mediante offsets constantes y despachar llamadas virtuales sin necesidad de recorrer la jerarquía.

Protocolos, iterables y el bucle for

El sistema de tipos de HULK incluye soporte para protocolos como mecanismo de abstracción nominal. Durante la fase de registro de símbolos, el `TypeChecker` almacena cada definición de protocolo en la tabla `protocols`, junto con la firma de sus métodos y la referencia a su protocolo padre. La conformidad de una clase a un protocolo se verifica mediante el método `type_conforms_to_protocol`, que comprueba que la clase implemente todos los métodos requeridos con las firmas adecuadas, respetando las reglas de

varianza: los tipos de los parámetros son contravariantes (la clase puede aceptar parámetros más generales que los del protocolo) y el tipo de retorno es covariante (puede devolver un subtipo más específico). Este mecanismo permite que el compilador acepte usos de protocolos en anotaciones de variables, parámetros y retornos, y se apoya en la restricción `ConformsToProtocol` durante la inferencia para posponer la verificación hasta que la variable de tipo se ligue a una clase concreta.

Un caso particularmente relevante es el de los iterables, que responden a la sintaxis `T*` —azúcar sintáctico para el tipo `Iterable(T)`. El compilador registra internamente el protocolo base `Iterable`, que declara los métodos `next(): Boolean` y `current(): Object`. Durante el análisis de un tipo definido por el usuario, si la clase implementa ambos métodos, se extrae el tipo de retorno de `current()` y se registra en un mapa `iterable_implementations` que asocia ese tipo de elemento con el nombre de la clase. Este registro se realiza al final del procesamiento de la clase, una vez que los métodos han sido completamente tipados.

Cuando el analizador encuentra una expresión o anotación de tipo `Iterable(T)`, aplica la función `replace_iterable_with_concrete`: si existe una clase registrada cuyo método `current()` devuelva exactamente `T`, la sustituye por `Class(nombre)`; en caso contrario, genera automáticamente un protocolo especializado `Iterable_T` que extiende `Iterable` y redefine `current(): T`. Esta transformación se aplica en todos los lugares donde se asigna un tipo a un nodo del AST —tipos de variables, parámetros, retornos, atributos, expresiones y anotaciones—, de modo que el código generado en fases posteriores nunca recibe un tipo genérico `Iterable` sin resolver.

El bucle `for` se procesa en el método `visit_for` siguiendo dos pasos. Primero, se evalúa el tipo de la expresión iterable. Si ese tipo conforma al protocolo `Enumerable` —definido por el usuario y que exige el método `iter(): Iterable`—, el iterable se reescribe como una llamada a `iterable.iter()`, transformando así un `for` sobre una colección múltiple en un `for` sobre el iterador que esta produce. En segundo lugar, se verifica que el iterable resultante sea una clase concreta que implemente los métodos `next()` y `current()`; el tipo de los elementos se obtiene del retorno de `current()` (resuelto y, si procede, convertido de `Iterable` a su clase concreta) y se declara una variable en el ámbito del bucle con ese tipo. El tipo global del bucle es el del cuerpo, que se analiza en el nuevo ámbito.

Finalmente, el aplanamiento de la jerarquía no necesita conocer la existencia de iterables ni protocolos, pues todos los tipos `Iterable` ya han sido sustituidos por clases o protocolos concretos en el AST. La fase de aplanamiento trabaja exclusivamente con las estructuras `FlattenedType` construidas a partir de las clases, y los métodos que implementan iteradores o enumeradores se tratan como cualquier otro método, garantizando que la generación de código sea uniforme y no requiera lógica especial para estas construcciones sintácticas.

Fases del análisis y gestión de errores

El análisis semántico se ejecuta a través del método `check`, que orquesta cuatro fases secuenciales. En la primera, se registran todos los símbolos globales —funciones, tipos y protocolos— en las tablas correspondientes, se generan variables de tipo frescas para los parámetros de las funciones, y se recorre el cuerpo de cada función y método aplicando unificación en cada expresión. En la segunda, una pasada de resolución sustituye cada variable de tipo que haya sido ligada por su tipo concreto, dejando el AST completamente tipado. La tercera fase invoca el aplanamiento de la jerarquía, construyendo las estructuras

FlattenedType para todas las clases. Finalmente, una cuarta pasada elimina cualquier variable de tipo residual que pudiera haber quedado en las expresiones de inicialización de los atributos.

Durante todas estas fases, el **TypeChecker** acumula los errores detectados en un vector en lugar de detenerse ante el primero. Cuando una expresión resulta inválida, se registra el diagnóstico correspondiente y se le asigna el tipo **Error**, que actúa como absorbente: las operaciones posteriores que lo involucren simplemente se ignoran o propagan el error sin generar nuevos mensajes. Este mecanismo evita las cascadas de diagnósticos redundantes y permite al programador recibir un informe completo de los problemas del programa en una sola ejecución.

3.5. Monomorfización de funciones genéricas

Una vez finalizado el análisis semántico, todas las expresiones del programa poseen información de tipos inferidos. Sin embargo, las funciones que fueron marcadas como genéricas —aquellas cuyos parámetros no pudieron ser vinculados a un tipo concreto— todavía contienen variables de tipo en su firma y en las anotaciones de su cuerpo. Puesto que el backend LLVM requiere tipos concretos para generar código, es necesario un pase adicional que elimine la genericidad antes de la traducción final. Este pase es la **monomorfización**.

La monomorfización consiste en transformar cada función genérica en una o varias versiones concretas, especializadas para los tipos con los que realmente es utilizada en el programa. El proceso se inspira directamente en el modelo de plantillas de **C++** y en el sistema de *generics* de **Rust**: en ambos lenguajes, cada instanciación con tipos distintos produce una nueva copia de la función, lo que permite que el compilador optimice agresivamente el código resultante sin necesidad de despacho dinámico ni comprobaciones de tipo en tiempo de ejecución.

En nuestra implementación, el pase de monomorfización se implementa mediante el **patrón Visitor**, que permite recorrer de forma uniforme todos los nodos del AST. El núcleo del algoritmo se encuentra en el método `visit_call`, responsable de especializar las llamadas a funciones genéricas. Su funcionamiento puede resumirse en los siguientes pasos:

1. **Visita recursiva de los argumentos.** Antes de procesar la llamada actual, se recorren recursivamente todos sus argumentos para que cualquier llamada genérica anidada sea especializada previamente.
2. **Detección de funciones genéricas.** Se consulta la información añadida durante el análisis semántico para determinar si la función invocada fue marcada como genérica. Si no lo es, la llamada permanece sin modificaciones.
3. **Obtención de la instancia concreta.** Si la función es genérica, se recolectan los tipos concretos de los argumentos — información ya disponible en las anotaciones del AST— y se genera un nombre único para la especialización mediante un proceso de *name mangling*.
4. **Crear la especialización.** Si la especialización requerida aún no ha sido generada:
 - a) Se **clona** la definición original de la función genérica.

- b) Se **sustituyen todas las variables de tipo** por los tipos concretos. Esta sustitución se aplica a los parámetros, al tipo de retorno y a cada uno de los nodos del cuerpo que contienen anotaciones de tipo, utilizando un mapa que asocia cada variable de tipo con su tipo real.
 - c) La copia resultante se **marca como no genérica** y se somete recursivamente al mismo proceso de monomorfización, para especializar cualquier llamada genérica que aparezca en su interior.
 - d) La especialización se **almacena** para su reutilización futura, evitando duplicar código cuando una misma función genérica se instancia con los mismos tipos en distintos puntos del programa.
5. **Reescritura la llamada original.** El nombre de la función en el nodo de llamada se modifica para que apunte a la nueva versión concreta, utilizando el nombre generado en el paso 3.

Tras finalizar el recorrido completo del programa, las definiciones genéricas originales se eliminan del AST, ya que todas las llamadas apuntan ahora a versiones especializadas.

Para evitar la generación infinita de especializaciones en presencia de funciones genéricas recursivas, el pase mantiene un conjunto de especializaciones que se encuentran en proceso de construcción. Si durante la instanciación se intenta generar nuevamente una especialización que ya está siendo procesada, el compilador detecta el ciclo y emite un error de compilación.

El resultado final es un AST completamente monomórfico, en el que todas las funciones poseen tipos concretos. De este modo, el backend puede generar código sin necesidad de conocer la existencia del polimorfismo paramétrico, convirtiendo la genericidad en un mecanismo puramente de tiempo de compilación.

3.6. Generación de código

La etapa final del compilador tiene como objetivo traducir el árbol de sintaxis abstracta (AST) a código intermedio LLVM IR. En este punto toda la información necesaria para la generación ya ha sido resuelta, incluyendo los tipos de las expresiones, la vinculación de identificadores y la estructura completa de la jerarquía de herencia, por lo que el backend puede concentrarse exclusivamente en producir una representación eficiente del programa.

La generación de código se implementó utilizando LLVM como infraestructura de compilación y la biblioteca *Inkwell*, que proporciona una interfaz segura para acceder a la API de LLVM desde Rust.

A continuación se describe la organización general del backend, la estrategia empleada para generar el código intermedio y las principales decisiones de diseño adoptadas para representar las características propias del lenguaje HULK.

Infraestructura del generador

Toda la generación de código se centraliza en la estructura `LlvmCodeGen`, responsable de mantener el estado necesario durante la traducción del programa. Esta estructura encapsula los principales componentes proporcionados por LLVM: el *Context*, encargado de administrar los tipos y demás objetos compartidos; el *Module*, que almacena todas las definiciones globales del programa, incluyendo funciones, estructuras, constantes y

variables globales; y el *Builder*, utilizado para insertar secuencialmente las instrucciones LLVM IR dentro del bloque básico activo. Adicionalmente, mantiene diversas estructuras auxiliares que facilitan la generación de código.

La traducción del programa se realiza mediante el patrón *Visitor*, empleado también en las etapas anteriores del compilador. La lógica de generación comienza en el nodo raíz **Program**, el cual coordina las distintas fases necesarias para construir el módulo LLVM.

Declaración de funciones y métodos

El backend declara todas las funciones y métodos de los tipos definidos por el usuario antes de generar sus respectivos cuerpos. Esta separación constituye una decisión de diseño necesaria debido a que HULK permite invocarlos independientemente del orden en que aparecen escritos en el código fuente.

Durante esta primera pasada únicamente se construyen las firmas LLVM de cada función, especificando su nombre, tipo de retorno y tipos de los parámetros. En el caso de los métodos, además se incorpora implícitamente un primer parámetro correspondiente al objeto receptor (**self**), representado como un puntero a la estructura del tipo al que pertenece el método. Cada método se registra utilizando un nombre completamente calificado de la forma **Tipo.método**, evitando conflictos entre métodos con el mismo nombre definidos en clases diferentes.

Además de las funciones definidas por el usuario, durante esta etapa también se incorporan las funciones predefinidas del lenguaje (*built-ins*). Algunas de estas funciones, como las matemáticas **sin**, **cos** o **sqrt**, se declaran de forma idéntica a las funciones de usuario y se invocan mediante una búsqueda genérica en el módulo. Otras, como **rand**, **log**, **print** o **range**, requieren una lógica de generación más compleja; para ellas se implementan *helpers* específicos que son invocados directamente desde **visit_call**, evitando así un tratamiento excepcional en el flujo principal de búsqueda y manteniendo el código modular.

Una vez que todas las declaraciones se encuentran disponibles, el backend realiza una segunda pasada para generar el cuerpo de cada función y método. De esta forma, cualquier llamada puede resolverse directamente consultando las declaraciones previamente registradas, sin depender del orden en que fueron definidas en el programa fuente.

Representación de objetos

La representación de los objetos constituye una de las principales decisiones del backend, ya que LLVM no incorpora de forma nativa un modelo orientado a objetos. En consecuencia, fue necesario definir una representación explícita capaz de soportar herencia, acceso a atributos y despacho dinámico de métodos.

Cada tipo definido por el usuario se traduce a una estructura (*struct*) de LLVM. La disposición de sus campos se obtiene a partir de la representación aplanada (*FlattenedType*) construida durante el análisis semántico. De esta forma, el backend únicamente reutiliza dicha información para construir la estructura correspondiente, sin volver a resolver aspectos relacionados con la herencia.

Como primer campo de toda estructura se incorpora un puntero a la tabla virtual (VTable), mientras que los atributos ocupan las posiciones restantes de la estructura.

Tablas virtuales

Cada tipo posee una única VTable global, compartida por todas sus instancias. Esta estructura concentra toda la información dinámica asociada al tipo, evitando duplicarla en cada objeto.

La VTable diseñada para HULK está compuesta por tres campos:

- Un identificador entero único (**type_id**) asignado durante el análisis semántico.
- Un puntero a la VTable del padre directo.
- Un arreglo de punteros a funciones correspondiente a los métodos efectivos del tipo.

Los dos primeros campos implementan el mecanismo de *identificación dinámica de tipos*. El identificador permite reconocer el tipo concreto de un objeto en tiempo de ejecución, mientras que el enlace hacia la VTable del padre posibilita recorrer la jerarquía de herencia cuando es necesario. Esta información es utilizada para implementar las operaciones **is** y **as**, las cuales determinan si un objeto pertenece a un determinado tipo o a cualquiera de sus ancestros.

La construcción de la VTable en el backend sigue un proceso determinista que refleja directamente el orden de los campos antes descrito. En primer lugar se emite el **type_id** como una constante entera de 32 bits, cuyo valor fue asignado durante el aplanamiento de tipos. A continuación se resuelve el puntero a la VTable del padre: si la clase hereda de otra, se recupera la VTable del ancestro directo (que ya ha sido generada gracias a que se ordenan topológicamente las clases previamente); en caso contrario, se utiliza un puntero nulo.

Finalmente se construye el arreglo de métodos. Para ello se ordenan los métodos aplanados según su índice de VTable (**vtable_index**) y, para cada uno, se obtiene el **FunctionValue** previamente declarado utilizando el nombre calificado **Tipo.metodo**. Este valor se inserta en la posición que le corresponde dentro del arreglo. Una vez ensamblados los tres campos, se crea una constante global de LLVM con el inicializador completo y se almacena para que todas las instancias del tipo puedan referenciarla a través del puntero situado en el primer campo de su estructura.

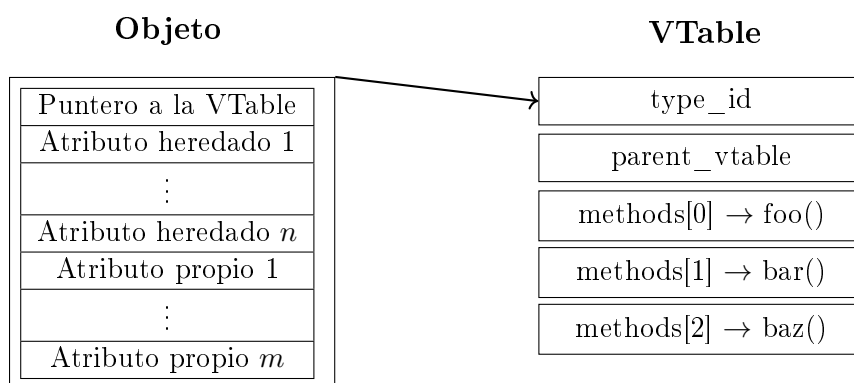


Figura 3.2: Relación entre un objeto y su tabla virtual.

Generación del punto de entrada

Una vez contruidos los tipos, declaradas las funciones y generados los cuerpos correspondientes, el backend crea la función de entrada del programa (**main**), que constituye el

punto de inicio de la ejecución.

A partir de este punto, la traducción continúa siguiendo el recorrido del árbol de sintaxis abstracta, donde cada nodo produce las instrucciones LLVM IR correspondientes.

Las expresiones aritméticas, estructuras de control, llamadas a funciones, creación de objetos y demás construcciones del lenguaje se traducen de forma incremental durante este recorrido, aprovechando las abstracciones ofrecidas por LLVM para representar operaciones, bloques básicos, control de flujo y acceso a memoria. De esta forma, el backend genera progresivamente el módulo LLVM completo que posteriormente podrá ser optimizado y compilado para la arquitectura destino.

Traducción del bucle `for`

El bucle `for` de HULK se transpila a una combinación de `let` y `while` que sigue el protocolo `Iterable`. Para una expresión `for (x in iterable) cuerpo`, la transformación equivale a:

```
let _iter = iterable in
  while (_iter.next())
  let x = _iter.current() in
  cuerpo
```

Esta traducción se realiza directamente en el backend durante la generación de código, sin necesidad de modificar el AST. El bucle resultante invoca dinámicamente los métodos `next()` y `current()` a través de la tabla de métodos virtuales (VTable), lo que permite iterar cualquier objeto que implemente el protocolo `Iterable`, independientemente de su tipo concreto. El valor de retorno del `for` es el de la última ejecución del cuerpo, igual que en un `while` convencional.

Soporte para el protocolo `Enumerable`

Para que una colección pueda recorrerse múltiples veces, HULK define el protocolo `Enumerable`, que añade el método `iter()`. Cuando un objeto implementa este protocolo, el `for` ya no lo itera directamente, sino que primero obtiene un nuevo iterador. La especificación del lenguaje establece la siguiente transformación:

```
let enumerable = <expresión> in
let iterable = enumerable.iter() in
while (iterable.next())
let x = iterable.current() in
cuerpo
```

El backend incorpora esta extensión mediante una comprobación previa al bucle: si el tipo del objeto posee el método `iter`, emite una llamada virtual a `iter()` y el iterador resultante reemplaza al objeto original. De este modo, colecciones como los vectores pueden ser iteradas repetidamente sin modificar la lógica básica del `for`.

La función `range` como iterable predefinido

La función `range(inicio, fin)` es una construcción built-in que devuelve un objeto iterable. Para integrarla con el mecanismo de despacho virtual, el analizador semántico inyecta automáticamente el tipo interno `_Range`, cuyos atributos (`min`, `max`, `current`) y métodos (`next()`, `current()`) cumplen el protocolo `Iterable`. La llamada a `range` se traduce a la creación de una instancia de `_Range`, inicializando sus campos con los argumentos recibidos.

Este diseño permite que `range` se beneficie del mismo mecanismo de iteración que cualquier otra clase definida por el usuario, sin requerir un tratamiento especial en el backend más allá de la construcción inicial del objeto.

3.7. Compilación a código nativo

Una vez que el backend ha completado la generación del módulo LLVM, éste se escribe en el archivo `output.ll`. A continuación, el compilador invoca a `clang` para traducir el código intermedio a un ejecutable nativo. Durante esta compilación se enlaza la biblioteca matemática (`-lm`) con el fin de resolver las funciones externas empleadas por el lenguaje, como las trigonométricas, la exponencial o la potencia.

Si el proceso finaliza sin errores, se obtiene un binario llamado `output` listo para ser ejecutado directamente.

3.8. Gestión de errores

El compilador sigue un modelo de detección temprana: cada etapa se ejecuta de forma secuencial y, antes de pasar a la siguiente, verifica que no se hayan producido errores. Si una fase encuentra problemas, el proceso se detiene y los errores se reportan al programador. Cada error incluye la ubicación exacta en el código fuente (línea y columna), una categoría que identifica la fase en que se originó —léxica, sintáctica, semántica o de generación de código— y un mensaje descriptivo que explica la causa del fallo. Esta organización permite ofrecer diagnósticos precisos y evita que errores tempranos se propaguen a etapas posteriores, donde resultarían más difíciles de interpretar.

Conclusiones

El presente trabajo ha abordado el lenguaje HULK desde una doble perspectiva: como objeto de análisis crítico y como meta de un proceso completo de compilación. Del estudio realizado se desprende que HULK ocupa un punto deliberadamente intermedio entre los paradigmas de programación más asentados. De la tradición imperativa de los lenguajes tipo Turing conserva la mutabilidad controlada, las estructuras de iteración clásicas y un modelo de objetos con herencia nominal. Del paradigma funcional, propio de los lenguajes tipo Church, toma la orientación a expresiones, la sintaxis `let...in` y un sistema de inferencia de tipos que reduce la necesidad de anotaciones explícitas.

Las extensiones incorporadas durante el proyecto abordan tres de las limitaciones más significativas del diseño base. El polimorfismo paramétrico en funciones, implementado mediante monomorfización, permite reutilizar una misma definición con múltiples tipos sin sacrificar el rendimiento, acercando a HULK al comportamiento de lenguajes como C++ o Rust. Los protocolos, por su parte, introducen una forma de tipado estructural que complementa el modelo nominal de clases y abre la puerta a un polimorfismo más flexible. Finalmente, el sistema de iterables —basado en los protocolos `Iterable` y `Enumerable`— generaliza el bucle `for` para que pueda recorrer cualquier tipo definido por el usuario que implemente el protocolo correspondiente, superando la restricción original que limitaba su uso a la función `range`. Las tres extensiones se han diseñado respetando la filosofía original del lenguaje y manteniendo la coherencia con su sistema de tipos.

La construcción del compilador ha permitido recorrer de forma integradora todas las fases clásicas de la traducción de lenguajes, desde la construcción manual de un analizador léxico basado en autómatas finitos hasta la generación de código nativo mediante LLVM. La naturaleza de HULK deja abiertas numerosas vías de extensión futura, como la incorporación de vectores, funciones anónimas o un sistema de inferencia más completo, para las que la arquitectura actual del compilador está preparada. En última instancia, el proyecto ha demostrado que HULK es un ecosistema de complejidad graduable que permite aprender, enseñar e investigar sobre compilación a muy distintos niveles.

Bibliografía

- [1] Alejandro Piad Morffis, *Principles of Programming Languages Design and Implementation: Building a HULK Interpreter and Compiler*, 2026. <https://matcom.github.io/hulk/>
- [2] Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman, *Compilers: Principles, Techniques, and Tools*.